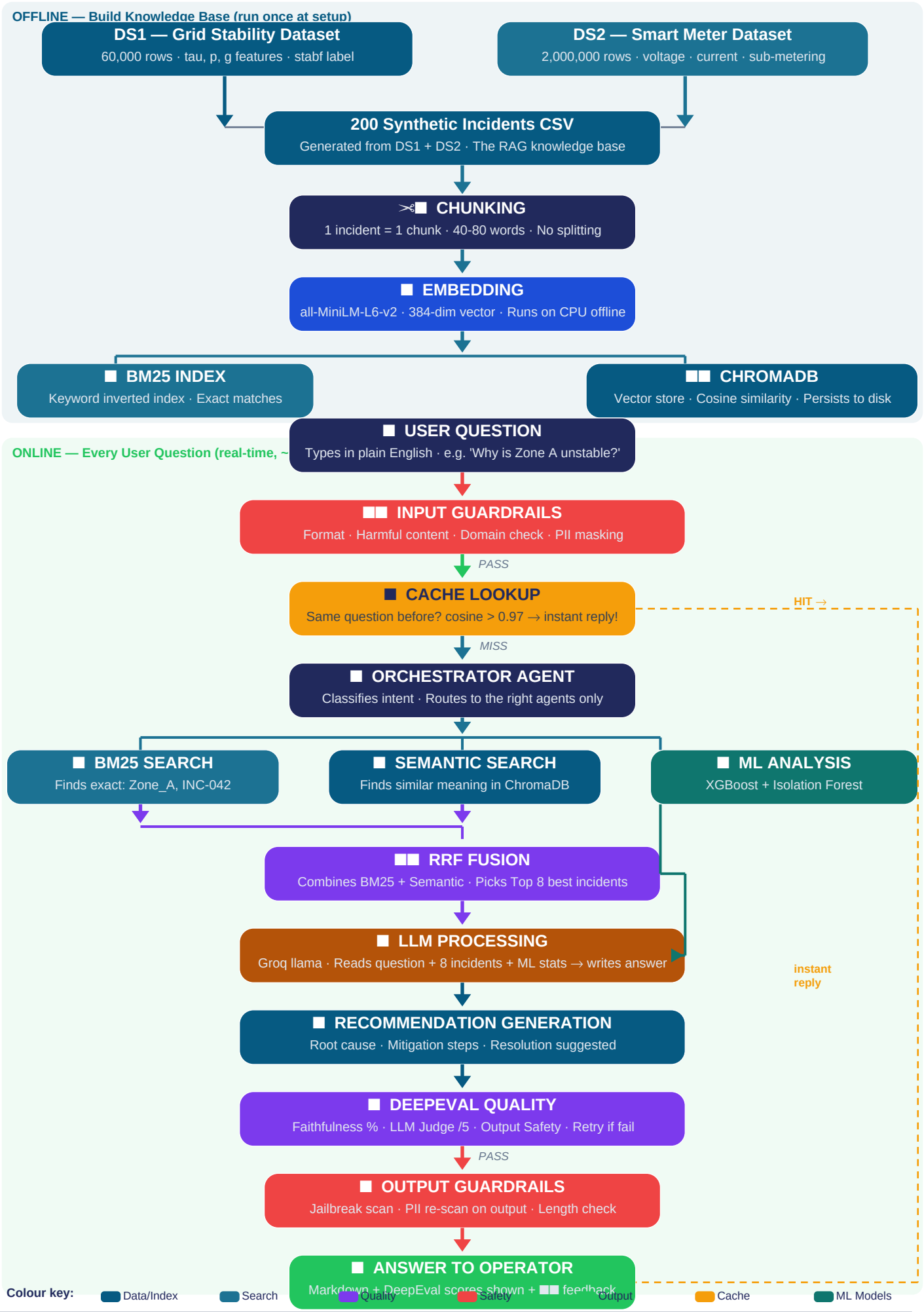
SGEIA — RAG Pipeline

Smart Grid Energy Intelligence Assistant


OFFLINE — Build Knowledge Base (run once at setup)
DS1 — Grid Stability Dataset
60,000 rows · tau, p, g features · stabf label
DS2 — Smart Meter Dataset
2,000,000 rows · voltage · current · sub-metering
200 Synthetic Incidents CSV
Generated from DS1 + DS2 · The RAG knowledge base
CHUNKING
1 incident = 1 chunk · 40-80 words · No splitting
EMBEDDING
all-MiniLM-L6-v2 · 384-dim vector · Runs on CPU offline
BM25 INDEX
Keyword inverted index · Exact matches
CHROMADB
Vector store · Cosine similarity · Persists to disk
ONLINE — Every User Question (real-time, ~
USER QUESTION
Types in plain English · e.g. 'Why is Zone A unstable?'
INPUT GUARDRAILS
Format · Harmful content · Domain check · PII masking
PASS
CACHE LOOKUP
Same question before? cosine > 0.97 → instant reply!
HIT →
MISS
ORCHESTRATOR AGENT
Classifies intent · Routes to the right agents only
BM25 SEARCH
Finds exact: Zone_A, INC-042
SEMANTIC SEARCH
Finds similar meaning in ChromaDB
ML ANALYSIS
XGBoost + Isolation Forest
RRF FUSION
Combines BM25 + Semantic · Picks Top 8 best incidents
LLM PROCESSING
Groq llama · Reads question + 8 incidents + ML stats → writes answer
instant reply
RECOMMENDATION GENERATION
Root cause · Mitigation steps · Resolution suggested
DEEPEVAL QUALITY
Faithfulness % · LLM Judge /5 · Output Safety · Retry if fail
PASS
OUTPUT GUARDRAILS
Jailbreak scan · PII re-scan on output · Length check
ANSWER TO OPERATOR
Markdown + DeepEval scores shown + feedback
Colour key:
Data/Index
Search
Quality
Safety
Output
Cache
ML Models

LangGraph StateGraph — Where & How It Is Used

File: src/agents/graph.py | **Function:** build_graph() | **Called by:** run_query() inside every /api/query and /api/chat request | **Purpose:** Orchestrates all 5 AI agents so they share data and run in the right order.

1

StateGraph(AgentState) — All Agents Share One Memory Object

AgentState is a TypedDict (src/agents/state.py) that holds: query, routing_decision, retrieved_incidents, health_score, anomaly_flags, root_cause, mitigation_steps, final_response. When Agent 1 classifies the query, Agent 2 reads that classification. When Agent 3 finds incidents, Agent 5 reads them. No agent re-does work already done.

2

Conditional Routing — Only the Right Agents Run

After the LLM Query Router classifies the question, LangGraph picks the path automatically: 'incident' → Grid Retrieval Agent only · 'stability' → Grid Stability Agent only · 'smart_meter' → Smart Meter Agent only · 'cross_domain' → all three agents run in parallel. This is done using add_conditional_edges() in graph.py.

3

MemorySaver Checkpoint — Conversation Memory Across Turns

checkpointer = MemorySaver() is passed to workflow.compile(). This saves the full AgentState after every question. If the operator asks a follow-up ('tell me more about Zone A'), the next turn has full previous context. Without MemorySaver: every question treated as brand new. With MemorySaver: full conversation continuity within a session.

4

5 Nodes Connected by Edges — The Fixed Pipeline Order

START → route_query → [retrieve_incidents / assess_stability / analyse_smart_meter] → analyse_failure → generate_recommendations → synthesise_response → END. Each node = one Python function = one agent. The output of each node writes to AgentState which the next node reads.

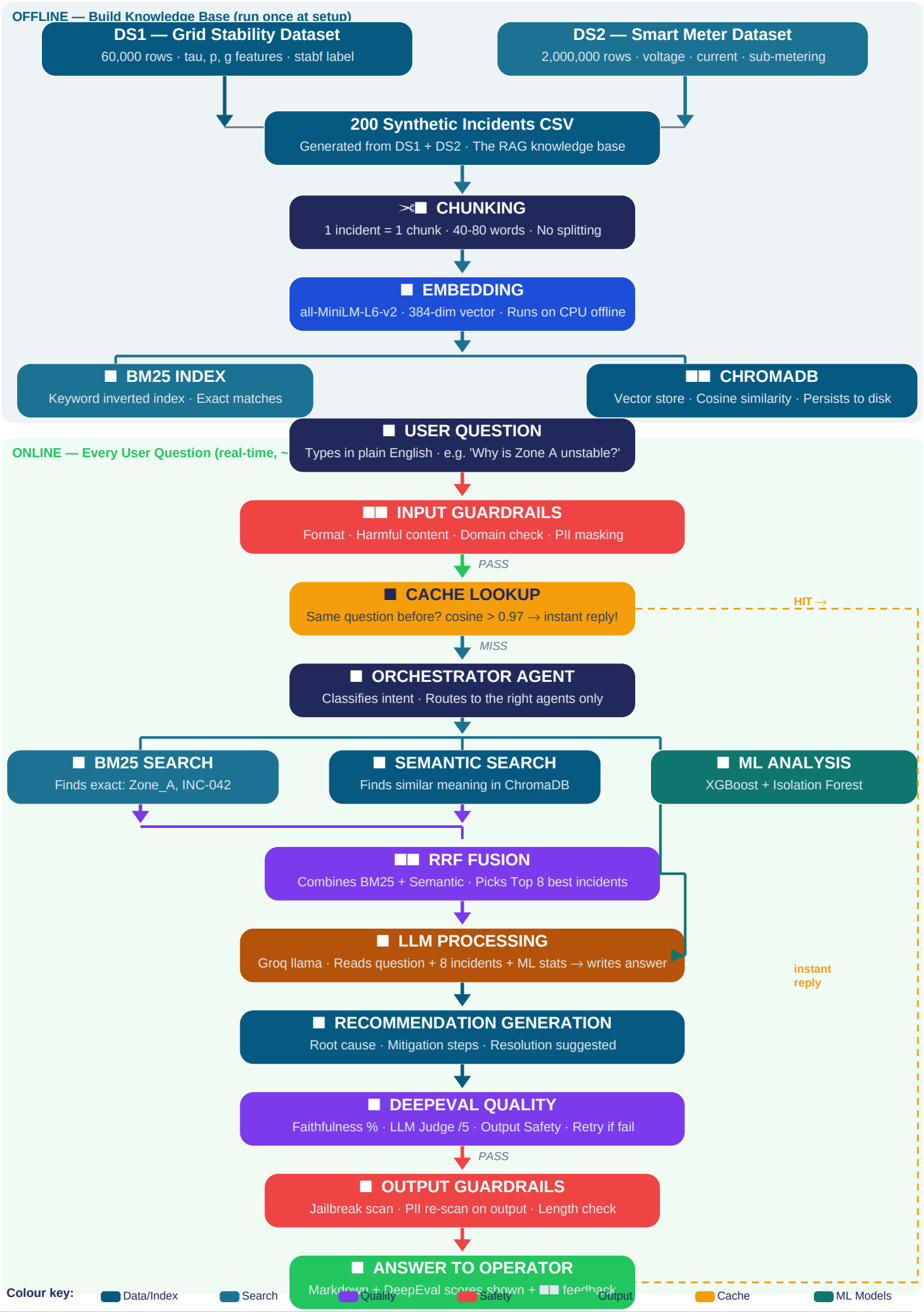
5

run_query() — Called for Every User Question

run_query(query, session_id, widget_context) calls app.invoke(initial_state, config={thread_id: session_id}). LangGraph executes the full graph. The final_response field in AgentState is returned to the FastAPI endpoint which sends it back to the browser.

Agent Node	File	Reads from AgentState	Writes to AgentState
route_query	query_router.py	query, widget_context	routing_decision, extracted_intent, metadata_filters
retrieve_incidents	retrieval_agent.py	query, metadata_filters	retrieved_incidents, retrieval_count
assess_stability	stability_agent.py	query	health_score, stability_label, stability_prob, anomaly_flags
analyse_smart_meter	smart_meter_agent.py	query	anomaly_count, anomaly_flags, meter_summary
retrieve_and_assess	graph.py (fan-out)	query (cross_domain)	All retrieval + stability + meter outputs
analyse_failure	failure_agent.py	retrieved_incidents, health_score	root_cause (cause, evidence, affected_zones)
generate_recommendations	recommendation_agent.py	root_cause, retrieved_incidents	mitigation_steps, judge_score, faithfulness_score
synthesise_response	graph.py	All AgentState fields	final_response (complete answer string)

RAG Pipeline — Complete Flow (Reference Copy)


OFFLINE — Build Knowledge Base (run once at setup)
DS1 — Grid Stability Dataset
60,000 rows · tau, p, g features · stabf label
DS2 — Smart Meter Dataset
2,000,000 rows · voltage · current · sub-metering
200 Synthetic Incidents CSV
Generated from DS1 + DS2 · The RAG knowledge base
CHUNKING
1 incident = 1 chunk · 40-80 words · No splitting
EMBEDDING
all-MiniLM-L6-v2 · 384-dim vector · Runs on CPU offline
BM25 INDEX
Keyword inverted index · Exact matches
CHROMADB
Vector store · Cosine similarity · Persists to disk
ONLINE — Every User Question (real-time, ~
USER QUESTION
Types in plain English · e.g. 'Why is Zone A unstable?'
INPUT GUARDRAILS
Format · Harmful content · Domain check · PII masking
PASS
CACHE LOOKUP
Same question before? cosine > 0.97 → instant reply!
HIT →
MISS
ORCHESTRATOR AGENT
Classifies intent · Routes to the right agents only
BM25 SEARCH
Finds exact: Zone_A, INC-042
SEMANTIC SEARCH
Finds similar meaning in ChromaDB
ML ANALYSIS
XGBoost + Isolation Forest
RRF FUSION
Combines BM25 + Semantic · Picks Top 8 best incidents
LLM PROCESSING
Groq llama · Reads question + 8 incidents + ML stats → writes answer
instant reply
RECOMMENDATION GENERATION
Root cause · Mitigation steps · Resolution suggested
DEEPEVAL QUALITY
Faithfulness % · LLM Judge /5 · Output Safety · Retry if fail
PASS
OUTPUT GUARDRAILS
Jailbreak scan · PII re-scan on output · Length check
ANSWER TO OPERATOR
Markdown + DeepEval scores shown + feedback
Colour key:
Data/Index
Search
Quality
Safety
Output
Cache
ML Models